

Note méthodologique

Preuve de Concept FLAN-T5

LAROZE-CERVETTI Antonine – 06/2025

Dataset retenu

Les données retenues sont les données utilisées pour le projet 6 de la formation Openclassrooms : « Classifier automatiquement des biens de consommations ».

Les données sont téléchargeables [ici](#)

Le dataset répertorie une liste de bien de consommation (1050) mis en vente sur un site internet.

Le dataset a été nettoyé pour l'entraînement du modèle FLAN-T5, à savoir :

- Chaque ligne représente un produit de consommation
- Deux variables sont gardées : la description du produit fait par l'utilisateur (text), ainsi que la catégorie à laquelle le produit appartient (label).
- Les catégories sont filtrées à partir de la colonne « *product_category_tree* », le 1^{er} niveau de catégorie est gardé pour simplification.
- Le nom des catégories a été harmonisés via un dictionnaire de correspondance pour être utilisables par les modèles.

Voici la liste des 7 catégories (sous le nom de « label ») :

1. baby_care
2. beauty_and_personal_care
3. computers
4. home_decor_and_festive_needs
5. home_furnishing
6. kitchen_and_dining
7. watches

La distribution des classes est parfaitement équilibrée (150 observations par catégories). Les données ont été prétraitées par les méthodes basiques de NLP : suppression des caractères spéciaux et chiffres, passage en minuscules, et suppression des espaces blancs. Enfin, le jeu a été divisé en ensembles d'entraînement (70%) , de validation (15%) et de test (15%) de manière aléatoire avec un seed fixe pour reproductibilité.

Les concepts de FLAN-T5

Le modèle FLAN-T5 est une version fine-tunée de T5 introduite par Google en 2022 dans le cadre du projet FLAN, Fine-tuned LAnguage Net, (Chung, et al., 2022)

Qu'est-ce qu'un modèle T5 ?

Un modèle T5 (Text-to-text-transfer transformer) est un modèle pré-entraîné basé sur l'architecture Transformers en mode « **encodeur-décodeur** ». Il a été développé par les équipes Google (plus particulièrement par les équipes Google brain) en 2020.

Le concept du modèle est que « tout problème de NLP (traduction, résumé, classification, question-réponse, etc.) est formulé comme une tâche de conversion de texte en texte », c'est-à-dire qu'il convertit toutes les tâches en une tâche de génération de texte. (Raffel, et al., 2020)

Par exemple pour une tâche de classification, comme utilisé ici, le modèle va générer les catégories. T5 est pré-entraîné sur un énorme corpus, appelé C4 (« *Colossal Clean Crawled Corpus* ») et utilise une tâche de pré-entraînement appelée span corruption¹. (Raffel, et al., 2020)

Qu'est-ce que l'architecture Transformers ?

L'architecture Transformers est une architecture de réseau de neurones introduite par (Vaswani, et al., 2017).

Contrairement aux réseaux de neurones récurrents (RNN) les Transformers permettent une parallélisation massive du traitement. En effet, les RNN traitent les données séquentiellement (lit mot par mot dans l'ordre), et ont tendance à oublier ou déformer le contexte lorsque les textes sont longs alors que les Transformers traitent les données en même temps, c'est le mécanisme d'« Attention » (Vaswani, et al., 2017).

Ce mécanisme, plus précisément de « Self-Attention », permet à chaque mot du texte d'interagir directement avec tous les autres mots du texte indépendamment de leur position (texte long ou non), ce qui permet au modèle de comprendre de manière plus optimale le contexte (Vaswani, et al., 2017).

Mais pour comprendre l'ordre des mots, qui est très important pour comprendre le contexte, les Transformers doivent être informés de la position des mots dans la séquence.

Deux stratégies principales existent :

- **Les Positional Encodings absolus**, utilisés dans le Transformer d'origine, attribuent à chaque mot une position fixe dans la phrase (0, 1, 2...). Ces positions sont transformées en vecteurs puis ajoutées à l'entrée. Cela permet au modèle de distinguer "*le chat dort*" de "*dort le chat*", mais le rend sensible aux longueurs fixes et à l'ordre absolu. (Vaswani, et al., 2017)

¹ Méthode pour apprendre au modèle à comprendre et générer du texte en remplaçant des spans (suite de mots) par des tokens (ids). Le modèle doit compléter les spans manquants dans le bon ordre.

- **Les Relative Position Embeddings**, utilisés dans T5, et donc dans FLAN-T5, indiquent les distances relatives entre les mots ("+1", "-1", etc.), au lieu d'une position fixe. Cela rend le modèle plus souple pour des séquences de longueurs variables, et mieux adapté à capter les relations contextuelles. Cela améliore sa capacité à généraliser. (Raffel, et al., 2020)

L'architecture du modèle est composée de deux modules principaux : un **encodeur** (qui traite les entrées) et un **décodeur** (qui génère les sorties).

Chaque bloc d'encodeur comprend trois couches principales :

- Un mécanisme de **Multi-Head Self-Attention** qui récolte les relations contextuelles en parallèle (plusieurs mots) ;
- **Un réseau feedforward** (afin de traiter les informations de la couche Self-Attention) ;
- Des opérations de normalisation (**LayerNorm**) et de régularisation pour éviter le surapprentissage (**Dropout**).

Le décodeur ajoute deux mécanismes supplémentaires :

- La **self-attention masquée** (ou causale), qui oblige le modèle à ne regarder que les mots précédents lors de la génération ;
- La **cross-attention**, qui permet de s'aligner sur toutes les représentations générées par l'encodeur.

Ensuite nous retrouvons les couches de réseau feedforward, de normalisation et de régularisation. Enfin le modèle génère les sorties grâce à une couche de prédiction (softmax). (Vaswani, et al., 2017); (Wolf, et al., 2020).

Pour résumé, le modèle T5 est donc un modèle basé sur l'architecture Transformers qui a été pré-entraîné sur un très grand nombre de corpus.

Le modèle FLAN-T5 est ,quant à lui, un modèle basé sur le modèle T5 (et donc l'architecture Transformers). FLAN-T5 se distingue de T5 par une phase de fine-tuning supplémentaire appelée "**instruction tuning**". Il comprend donc beaucoup mieux les instructions, parce qu'il a été entraîné sur des milliers de tâches formulées en langage naturel (Chung, et al., 2022).

Grâce à cela, FLAN-T5 excelle dans les scénarios zero-shot (le modèle doit résoudre une tâche sans avoir vu d'exemples) et few-shot (le modèle a vu seulement quelques exemples) (Chung, et al., 2022).

Modélisation

FLAN-T5 a été choisi pour sa performance reconnue sur une variété de tâches en langage naturel, en particulier la génération de texte conditionnelle.

Le modèle utilisé est le modèle FLAN-T5-SMALL via la bibliothèque Hugging Face Transformers. L'entraînement a été réalisé avec le Trainer de HuggingFace avec des arguments spécifiques définis par la classe Seq2SeqTrainingArguments.

Ici, l'objectif est que le modèle apprenne à bien classifier les produits dans la bonne catégorie, i.e. de générer la bonne catégorie pour le produit. La tâche est donc formulée sous forme d'une consigne explicite de génération de texte :

"Classify the product into one of the following categories EXACTLY AS WRITTEN: baby_care, beauty_and_personal_care, computers, home_decor_and_festive_needs, home_furnishing, kitchen_and_dining, watches. Output only the label from the list."

L'entraînement s'est déroulé dans un premier temps sur 10 epochs, ensuite sur 50 epochs pour capter le moment de convergence du modèle, avec les hyperparamètres principaux suivants :

- learning_rate = 3e-5 (taux d'apprentissage, plus il est bas plus l'exécution est lente)
- batch_size = 4 (nombre dans le sous-groupe)
- label_smoothing = 0.1 (ajout de flou pour éviter le surapprentissage et de surestimer les bons labels)
- weight_decay = 0.01 (sanction des poids utile quand peu d'observations comme ici)
- early_stopping = 4 epochs sans amélioration

Le critère d'arrêt (early_stopping) est basé sur la minimisation de la perte (loss), et le modèle est réévalué à chaque epoch sur le jeu de validation.

La tâche étant une classification textuelle, la métrique d'évaluation retenue est **l'accuracy**, définie comme le pourcentage de labels générés identiques aux labels d'origine.

À la fin de l'entraînement, le modèle est évalué sur le jeu de test. Les performances sont mesurées avec l'accuracy globale et le rapport de classification (précision, rappel, F1-score par classe).

Terme	Définition
Vrai positif (TP)	Le modèle prédit le produit dans la bonne classe. Le produit est une montre et c'est effectivement une montre.
Vrai négatif (TF)	Le modèle prédit que le produit n'est pas une montre et le produit n'est effectivement pas une montre.
Faux négatif (FN)	Le modèle prédit que le produit appartient à une autre classe. Le produit est une montre mais le modèle prédit une autre catégorie.
Faux positif (FP)	Le modèle prédit que c'est une montre alors que le produit appartient à une autre catégorie.

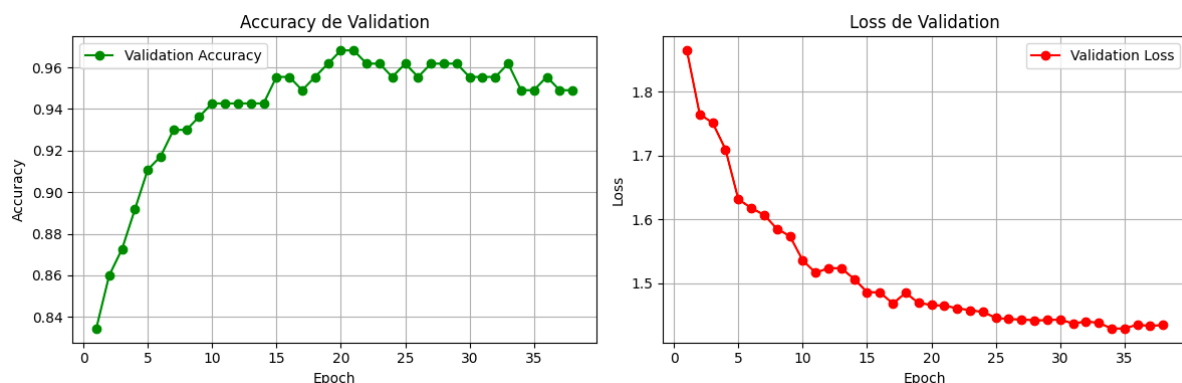
Précision	Dans toutes les prédictions que le modèle a classifié dans une catégorie, combien sont effectivement de cette catégorie. <i>Combien de produits classés dans montre sont effectivement des montres ?</i>
Rappel	Dans tous les produits d'une classe, combien le modèle a réussi à en détecter. <i>Combien de montres (effectivement montres) le modèle a réussi à classifier correctement ?</i>
F1 score	Moyenne harmonique entre précision et rappel. Si le F1-score est élevé, alors le modèle arrive à un bon équilibre entre ne pas se tromper de catégories et ne pas manquer de produits.

Résultats

Nous comparons ici le modèle FLAN-T5 avec un modèle de régression logistique sur des vecteurs TF-IDF (méthode qui permet de calculer le poids de mots rares vs fréquents).

1. Résultats sur la validation

L'entraînement se stoppe grâce au early stopping à 38 epochs pour une accuracy atteinte de 0.95. La meilleure epoch en terme d'accuracy est l'epoch 21 avec une accuracy atteinte de 0.97 (mais une loss un peu plus élevée que l'epoch 38). Ces scores sont atteints après plus de 12h d'entraînement.



2. Résultats sur le test

Flan-T5	precision	recall	F1-score	Support
baby_care	0.85	0.81	0.83	21
beauty_and_personal_care	1.00	0.93	0.97	15
Computers	1.00	1.00	1.00	30
home_decor_and_festive_needs	0.83	0.87	0.85	23
Home_furnishing	0.95	0.84	0.89	25
Kitchen_and_dining	0.86	1.00	0.93	19
Watches	0.96	1.00	0.98	25

Accuracy			0.92	158
Macro avg	0.92	0.92	0.92	158
Weighted avg	0.93	0.92	0.92	158

Le modèle prédit 92 % de produits dans la bonne catégorie, ce qui montre la performance élevée du modèle FLAN-T5. :

- Excellente performance sur *computers et watches* : F1-score parfait ou quasi-parfait (1.00 et 0.98), indiquant une prédiction systématiquement correcte pour ces classes.
- Les classes les plus difficiles pour le modèle sont *baby_care* (f1-score de 0.83) et *home_furnishing* (0.89), où il semble confondre certains exemples, comme l'indique le rappel plus bas.

3. Résultats Tf-IDF

Pour l'échantillon de validation, le modèle de régression logistique avec les vecteurs Tf-IDF atteint une accuracy de 0.95. L'accuracy est similaire du modèle FLAN-T5 mais est beaucoup plus rapide d'entraînement.

Régression logistique (TFIDF) Test	precision	recall	f1-score	Support
baby_care	0.89	0.81	0.85	21
beauty_and_personal_care	1.00	1.00	1.00	15
Computers	0.97	1.00	0.98	30
home_decor_and_festive_needs	0.87	0.87	0.87	23
Home_furnishing	0.92	0.96	0.94	25
Kitchen_and_dining	0.89	0.89	0.89	19
Watches	1.00	1.00	1.00	25
Accuracy			0.94	158
Macro avg	0.94	0.93	0.93	158
Weighted avg	0.94	0.94	0.94	158

Le modèle de régression logistique avec TF-IDF atteint une accuracy de 94 % sur le jeu de test, surpassant légèrement Flan-T5 (92 %) :

- Il excelle sur plusieurs classes, notamment *beauty_and_personal_care* et *watches*, pour lesquelles il obtient un f1-score parfait de 1.00, et quasi-parfait pour la classe *computers* ce qui indique une prédiction sans erreur.
- La classe *baby_care* reste, comme pour Flan-T5, la plus difficile à modéliser (F1 = 0.85), bien que la précision y soit plus élevée.

Bien que Flan-T5 soit un modèle très performant, capable de mieux capter la structure du langage, dans ce cas précis, la régression logistique avec TF-IDF offre des performances similaires voire meilleures à un coût computationnel inférieur, ce qui en fait un choix très compétitif pour cette tâche de classification.

Interprétabilité

L'objectif est d'expliquer la prédiction du modèle, c'est-à-dire d'expliquer quels mots ont impacté la classification dans un label ou un autre.

Pour la régression logistique via les vecteurs Tf-IDF, l'interprétabilité du modèle est effectuée dans le notebook facilement via les coefficients. L'interprétation globale et locale de ce modèle ne sera pas présentée ici pour se concentrer sur l'interprétabilité du modèle FLAN-T5.

Méthodologie pour FLAN-T5

Les modèles de type Transformer comme FLAN-T5 présentent une grande difficulté d'interprétation, car ils fonctionnent comme des boîtes noires, où il est difficile de savoir pourquoi une décision a été prise (Guidotti, et al., 2018); (Belinkov & Glass, 2019). Contrairement à des modèles linéaires ils n'offrent pas de coefficients directement interprétables.

J'ai donc adopté une approche Leave-One-Out (LOO), qui est un cadre simple mais puissant pour évaluer l'importance d'un mot (ou token) dans une prédiction.

Définition de LOO : La méthode LOO consiste à retirer un mot à la fois dans le texte d'entrée, à réexécuter le modèle, et à mesurer l'impact de cette suppression sur la sortie. Si la suppression d'un mot change fortement la prédiction (ou la loss), alors ce mot est considéré comme important (Mu, Pouransari, Kim, & al., 2023); (Belinkov & Glass, 2019)

La limite principale de cette méthode est qu'elle est très coûteuse en calcul et en temps d'exécution. (Belinkov & Glass, 2019)

Cette méthode est compatible avec les modèles text-to-text comme FLAN-T5, car elle repose uniquement sur l'entrée-sortie du système, sans supposer une architecture spécifique (comme pour SHAP ou LIME).

Application à FLAN-T5 :

1. Pour chaque exemple : génération de la prédiction complète à partir du texte d'origine.
2. Boucle sur chaque mot, en le supprimant, puis en régénérant la sortie via `model.generate()`.
3. Deux approches d'évaluation possibles :
 - Via la perte (loss) : calcule le score de perte du modèle avec et sans le mot retiré. La perte est calculée par rapport au label originel. Cela donne un score d'importance continue : plus la perte augmente, plus le mot est important. Cela permet une fine analyse, même si la classe ne change pas,

mais est plus difficile d'interprétation. Cette méthode a été testé dans le notebook mais ne sera pas présentée ici.

- Via la précision (accuracy) : vérifie si la classe prédite change après la suppression du mot. Si oui, c'est un indice fort que le mot est décisif. Le score est ici binaire (0 : pas de changement et 1 : changement) ce qui permet une interprétation beaucoup plus simple et directe mais moins fine. Ici le score est calculé entre la prédiction originelle et la prédiction perturbée.

Résultats sur une observation (local) :

Élément analysé	Valeur
Texte	<i>buy mandhania cotton bath towel at rs at flipkartcom only genuine products free shipping cash on delivery</i>
Prédiction	home_furnishing
Mots via accuracy	1 = Changement de label (mots importants): buy(1), mandhania(1), bath(1), cotton (0), towel(0)

La méthode LOO via accuracy montre que la suppression de certains mots (*buy, mandhania, bath*) fait changer la prédiction, ce qui signifie que ces mots exercent une influence importante sur la décision du modèle.

Résultats par classe (global)

Classe	Mots les plus influents
watches	key (1.00), model (1.00), dimensions (1.00)
baby_care	bornbabykids (1.00) ; birthday (1.00) ; bath (0.67)
home_decor_and_festive_needs	men (1.00), bed (0.50), towels (0.50)

Au niveau global, certains mots paraissent plus évidents que d'autres selon les classes (bornbabykids pour baby_care, towels ou beds pour home_decor vs key pour watches).

Limites et améliorations

Limites

- **Temps d'entraînement très élevé** : plus de 12 heures ce qui rend le modèle FLAN-T5 coûteux à utiliser en pratique.
- **Performances inférieures malgré sa complexité** : bien que puissant, le modèle n'a pas surpassé le modèle classique de régression logistique avec les vecteurs TF-IDF. Cela suppose que le modèle FLAN-T5 a été sous-utilisé et/ou que le nombre d'observations est trop faible.

- Bien que les vecteurs Tf-IDF ne saisissent pas le contexte, la base de données actuelle utilise des mots très discriminants dans les descriptions, ce qui améliore leur performance.
- **Interprétabilité plus difficile** : FLAN-T5 étant un modèle génératif, il ne donne pas naturellement d'indications sur les mots-clés ou les poids, contrairement à la régression logistique.

Pistes d'amélioration

- **Utilisation de modèles plus puissants** : FLAN-T5-base ou large pour avoir des résultats plus performants mais cela sous-entend une machine beaucoup plus puissante que celle utilisée ici.
- **Améliorer le prompt** : en laissant le modèle générer librement une catégorie (sans choisir parmi des classes fixes), on pourrait mieux exploiter la capacité de FLAN à suivre des instructions ouvertes.

Bibliographie

1. Belinkov, Y., & Glass, J. (2019). *Analysis Methods in Neural NLP: A Survey*. Lien URL : <https://arxiv.org/abs/1812.08951> MIT.
2. Chung, H. W., Hou, L., Longpre, S., Zoph, B., Tay, Y., Fedus, W., . . . al., e. (2022). *Scaling Instruction-Finetuned Language Models*. Lien URL : <https://arxiv.org/abs/2210.11416> Google DeepMind.
3. Guidotti, R., Monreale, A., Ruggieri, S., Turini, F., Giannotti, F., & Pedreschi, D. (2018). A Survey of Methods for Explaining Black Box Models. *ACM Computing Surveys (CSUR)* , Lien URL : <https://arxiv.org/abs/1802.01933>, Volume 51, Numéro 5, Pages: 1-42.
4. Mu, J., Pouransari, H., Kim, Y., & al., e. (2023). *AttriBoT: A Bag of Tricks for Efficiently Approximating Leave-One-Out Context Attribution in LLMs*. Lien URL : <https://arxiv.org/abs/2411.15102> Google DeepMind.
5. Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., . . . Liu, P. J. (2020). *Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer*. Lien URL : <https://arxiv.org/abs/1910.10683> Google Research.
6. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., . . . Polosukhin, I. (2017). Attention is All You Need. *Advances in Neural Information Processing Systems (NeurIPS)* (p. Lien URL : <https://arxiv.org/abs/1706.03762>) Long Beach, CA: NeurIPS.

7. Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., . . . al., e. (2020). Transformers: State-of-the-Art Natural Language Processing. *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations (EMNLP)* (p. Lien URL : <https://arxiv.org/abs/1910.03771>). Online: ACL.